

Linq Zip Method in C# with Example

In this article, I will discuss the **LINQ Zip Method in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the [LINQ Prepend Method](#) with an example. At the end of this article, you will understand the LINQ Zip method and the need and use of the LINQ Zip Method in C# with Examples.

LINQ Zip Method in C#:

In C#, the LINQ Zip method combines elements from two or more sequences or collections into a single sequence by pairing corresponding elements together. It allows you to create tuples or apply a specified function to combine elements based on their positions in the input sequences. The Zip method generates a new sequence whose length is determined by the shortest input sequence. The signature of this method is given below.

```
public static IEnumerable<TResult> Zip<TFirst, TSecond, TResult>(this IEnumerable<TFirst> first,
IEnumerable<TSecond> second, Func<TFirst, TSecond, TResult> resultSelector);
```

Type Parameters:

1. **TFirst**: The type of elements of the first input sequence.
2. **TSecond**: The type of elements of the second input sequence.
3. **TResult**: The type of elements of the result sequence.

Parameters:

1. **IEnumerable<TFirst> first**: The first sequence to merge.
2. **IEnumerable<TSecond> second**: The second sequence to merge.
3. **Func<TFirst, TSecond, TResult> resultSelector**: A function that specifies how to merge the elements from the two sequences.

Returns: This method will return an **IEnumerable<T>** that contains merged elements of two input sequences.

Exceptions: This method will throw **ArgumentNullException** when either the first or the second input sequence is null.

Note: The Zip method merges each element of the first sequence with an element in the second sequence with the same index position. If both the sequences do not have the same number of elements, then the Zip method merges sequences until it reaches the end of the sequence, which contains fewer elements. For example, if one sequence has five elements and the other has four elements, the result sequence will have only four elements.

Example to Understand LINQ ZIP Method using C#:

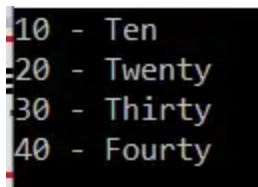
The following example shows how to merge two sequences using the Linq Zip method.

```

using System;
using System.Linq;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] numbersSequence = { 10, 20, 30, 40, 50 };
            string[] wordsSequence = { "Ten", "Twenty", "Thirty", "Fourty" };
            var resultSequence = numbersSequence.Zip(wordsSequence, (first, second) =>
first + " - " + second);
            foreach (var item in resultSequence)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}

```

Output:



```

10 - Ten
20 - Twenty
30 - Thirty
40 - Fourty

```

The first sequence contains 5 elements, whereas the second sequence contains 4 elements. So, for the fifth element of the first sequence, there is no corresponding fifth element in the second sequence. As a result, the Zip method merges the four elements, and that's what we have seen in the output.

Note: The Zip method is implemented by using deferred execution. So, the immediate return value of this method will be an object that stores all the required information required to perform the action. The query represented by this method is not executed until the object is enumerated either by calling its GetEnumerator method directly or by using for each loop.

Points to Remember While Working With LINQ Zip Method:

Key points to keep in mind when using the Zip method:

- The Zip method works best when you have multiple sequences of the same length that need to be combined element by element.
- If the input sequences have different lengths, the result sequence will have the length of the shortest input sequence and any excess elements in the longer sequences will be ignored.
- You can use the resultSelector function to specify how to combine elements from the input sequences. It can be a lambda expression or a delegate that takes elements from both sequences and produces elements for the result sequence.

- The Zip method is useful for scenarios where you need to create pairs, tuples, or combine data from multiple sources based on their positions.
- You can zip more than two sequences by chaining multiple Zip calls together.
- The result of the Zip method is an IEnumerable, and you can further query or manipulate the result using other LINQ methods.

When to Use Linq Zip Method in C#?

The LINQ Zip method is particularly useful in situations where you need to combine elements from two sequences in a pairwise manner. Here are several scenarios where Zip might be the appropriate choice:

- **Combining Data:** When you have two logically related collections, and you want to combine their elements, such as merging lists of names and corresponding email addresses.
- **Pairwise Operations:** For performing operations on two sequences simultaneously, like adding the corresponding elements of two lists of numbers.
- **Creating Tuples or Objects:** When you need to create a new sequence of tuples or objects that contain elements from each sequence, such as pairing indices with elements or combining properties from separate sequences into a single object.
- **Parallel Iteration:** If you want to iterate over two collections in parallel without an index, Zip provides a more elegant and functional approach than traditional for-loops.
- **Interleaving Sequences:** To interleave or weave two sequences together, Zip can be used to alternate elements from each sequence.
- **Data Transformation:** When transforming data from two sequences into a new form, like applying a function that takes two inputs and produces a single output.
- **Synchronization:** When you have two sequences that should be processed in lock-step, ensure that you are working with a pair of corresponding elements for each step.

Here's an example of using Zip to combine two lists into a dictionary:

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            var keys = new List<string> { "ID", "Name", "Email", "Mobile" };
            var values = new List<string> { "1", "Pranaya", "Pranaya@example.com", "1234567890" };
            var dictionary = keys.Zip(values, (k, v) => new { k, v })
                               .ToDictionary(x => x.k, x => x.v);
            // Now dictionary contains { { "ID", "1" }, { "Name", "Pranaya" },
            // { "Email", "Pranaya@example.com" }, { "Mobile", "1234567890" } }
            foreach (var item in dictionary)
            {
                Console.WriteLine($"{item.Key} - {item.Value}");
            }
            Console.ReadKey();
        }
    }
}
```

```
}  
}  
}
```

In this example, the Zip Method merges two lists into a sequence of anonymous types with properties k and v, then converts them into a dictionary.

Zip is convenient when you need to combine elements from each sequence into a single result, and it can lead to more readable and expressive code compared to traditional looping constructs. However, it's important to ensure the sequences are aligned correctly, as Zip will silently ignore extra elements in the longer list if the sequences are of unequal lengths.

In the next article, I will discuss [Linq ToList and ToArray Methods in C#](#) with Examples. In this article, I explain the LINQ ZIP in C# with Examples, and I hope you enjoy this LINQ ZIP in C# article.